WEEK 3 · LESSON 16 OF 24 · HANDOUT

Bias, Variance and Regularisation

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=EuBBz3bI-aA — open the lesson online to watch inside the course.

🎓 **Go deeper:** StatQuest: Regularization Part 1 — Ridge (L2) Regression — https://www.youtube.com/watch?v=Q81RR3yKn30

🎓 **Go deeper:** Stanford Machine Learning, Lecture 1 — Andrew Ng — https://www.youtube.com/watch?v=UzxYlbK2c7E

Your goal in this lesson: Recognise underfitting and overfitting from train and test errors, explain the bias–variance trade-off, and use Ridge and Lasso regularisation to tame complex models.

Two ways to be wrong

- **Bias** — the model is too **simple** to capture the pattern. It is wrong on training **and** test data. (**Underfitting.**)
- **Variance** — the model is too **flexible** and learns the noise. It is excellent on training data and poor on new data. (**Overfitting.**)

The best model sits in between: complex enough to learn the real pattern, simple enough to ignore the noise. This is the **bias–variance trade-off**.

See it with curves

Twenty noisy points follow a wave. We fit polynomials of degree 1 (a straight line), 3 and 15:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(2)
x = rng.uniform(0, 1, 20)
y = np.sin(2 * np.pi * x) + rng.normal(0, 0.3, 20)
x_new = rng.uniform(0, 1, 200)                      # new data from the same wave
y_new = np.sin(2 * np.pi * x_new) + rng.normal(0, 0.3, 200)

grid = np.linspace(0, 1, 300).reshape(-1, 1)
for degree in [1, 3, 15]:
    m = make_pipeline(PolynomialFeatures(degree), LinearRegression()).fit(x.reshape(-1, 1), y)
    train_mse = mean_squared_error(y, m.predict(x.reshape(-1, 1)))
    new_mse = mean_squared_error(y_new, m.predict(x_new.reshape(-1, 1)))
    print(f"degree {degree:2d}: train MSE {train_mse:.3f}, new-data MSE {new_mse:.3f}")
    plt.plot(grid, m.predict(grid), label=f"degree {degree}")
plt.scatter(x, y, color="black")
plt.ylim(-2, 2)
plt.legend()
plt.show()
```

Degree	Train error	New-data error	Diagnosis
1	high (about 0.28)	high (about 0.30)	Underfitting — high bias
3	low (about 0.08)	low (about 0.11)	Good balance
15	lowest (about 0.06)	very high (about 1.8)	Overfitting — high variance

The degree-15 curve wiggles through almost every training point and goes wild between them.

How to diagnose your own model

You see	Problem	Try
Train error high, test error high	Underfitting	A more flexible model, better features
Train error low, test error much higher	Overfitting	More data, a simpler model, **regularisation**
Both low and close	Good fit	Ship it (after the final test)

Regularisation: a penalty for complexity

Regularisation adds a penalty for big coefficients, so the model prefers simple explanations.

- **Ridge (L2)** — penalty = alpha × (sum of squared coefficients). Shrinks all coefficients towards zero.
- **Lasso (L1)** — penalty = alpha × (sum of absolute coefficients). Can push some coefficients **exactly to zero**, removing useless features.

alpha sets the strength: 0 means plain linear regression; larger values mean simpler models. Features must be **scaled** first, so the penalty treats them fairly.

```
ridge15 = make_pipeline(PolynomialFeatures(15), StandardScaler(), Ridge(alpha=0.1))
ridge15.fit(x.reshape(-1, 1), y)
print("Ridge degree 15, new-data MSE:",
      round(mean_squared_error(y_new, ridge15.predict(x_new.reshape(-1, 1))), 3))
```

The same degree-15 model, with a Ridge penalty, drops from about **1.8** to about **0.11** — as good as the degree-3 model.

Regularisation on the homes data

Now a realistic danger: only **40** homes to learn from, and 20 extra columns of pure random noise (in real projects, many columns turn out to be useless).

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 600
district = rng.choice(["Gasabo", "Kicukiro", "Nyarugenge"], n, p=[0.45, 0.35, 0.2])
bedrooms = rng.integers(1, 5, n)
size_m2 = (bedrooms * 28 + rng.normal(20, 12, n)).clip(25).round()
km_to_centre = rng.uniform(0.5, 15, n).round(1)
furnished = rng.random(n) < 0.3
base = {"Gasabo": 120_000, "Kicukiro": 90_000, "Nyarugenge": 100_000}
rent = (np.array([base[d] for d in district]) + 2_600 * size_m2 - 6_000 * km_to_centre
        + np.where(furnished, 60_000, 0) + rng.normal(0, 25_000, n)).round(-3)
homes = pd.DataFrame({"district": district, "bedrooms": bedrooms, "size_m2": size_m2,
                      "km_to_centre": km_to_centre, "furnished": furnished, "rent_rwf": rent})
print(homes.shape)
homes.head()
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_absolute_error

X = pd.get_dummies(homes.drop(columns="rent_rwf"), columns=["district"], dtype=int)
X["furnished"] = X["furnished"].astype(int)
noise = np.random.default_rng(1).normal(size=(len(X), 20))
X = pd.concat([X, pd.DataFrame(noise, columns=[f"noise_{i}" for i in range(20)])], axis=1)
y = homes["rent_rwf"]

X_small, X_rest, y_small, y_rest = train_test_split(X, y, train_size=40, random_state=42)
for name, reg in [("Linear", LinearRegression()), ("Ridge", Ridge(alpha=10)),
                  ("Lasso", Lasso(alpha=4000, max_iter=10_000))]:
    m = make_pipeline(StandardScaler(), reg).fit(X_small, y_small)
    print(f"{name:6s} train MAE {mean_absolute_error(y_small, m.predict(X_small)):8.0f}"
          f"   test MAE {mean_absolute_error(y_rest, m.predict(X_rest)):8.0f}")
```

- Plain linear regression: train MAE about **10,000**, test MAE about **48,500** — badly overfitted.
- Ridge: test MAE about **33,000**.
- Lasso: test MAE about **27,000** — and it switched off most of the noise columns completely (only about 6 of the 20 keep a non-zero coefficient). It also dropped `bedrooms`, which size already explains:

```
lasso = make_pipeline(StandardScaler(), Lasso(alpha=4000, max_iter=10_000)).fit(X_small,
y_small)
coefs = pd.Series(lasso[-1].coef_, index=X.columns)
print("Noise columns kept:", int((coefs.filter(like="noise") != 0).sum()), "of 20")
```

Regularisation is everywhere

- `LogisticRegression` is regularised by default; **C** is the inverse strength (small C = strong penalty).
- Trees are regularised by `max_depth` and `min_samples_leaf`; boosting by the learning rate; neural networks by dropout and weight decay.

Key terms

- **Bias** — error from a model that is too simple (underfitting).
- **Variance** — error from a model that is too sensitive to the training data (overfitting).
- **Regularisation** — a penalty that discourages complexity.

- **Ridge (L2)** — shrinks all coefficients.
- **Lasso (L1)** — shrinks and can set coefficients to zero (feature selection).
- **alpha / C** — the strength of the penalty (C is the inverse).

Remember this: Compare train and test errors to tell bias from variance; regularisation trades a little training accuracy for much better results on new data.

Name: _____ Date: _____

Activity

Task 1

Repeat the curve experiment with degrees 2, 5 and 9. Which degree gives the lowest new-data error?

Task 2

Try Ridge alphas 0.001, 0.1, 1 and 10 on the degree-15 model. What happens when alpha is too big?

Task 3

On the 40-home experiment, try Lasso alphas 1000, 4000 and 15000. Which features survive at 15000, and what happens to the test MAE?

Task 4

Train the 40-home models again with all 480 remaining homes as training data. Does regularisation still matter as much? Why?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. A model has high error on both training and test data. What is the problem?
   - A. Underfitting (high bias)
   - B. Too much data
   - C. Data leakage
   - D. Overfitting
2. A model has train MAE 10,000 and test MAE 48,500. What is the problem?
   - A. Perfect fit
   - B. Wrong units
   - C. Overfitting (high variance)
   - D. Underfitting
3. What does Lasso do that Ridge does not?
   - A. Only works for trees
   - B. Uses no penalty
   - C. Can set some coefficients exactly to zero
   - D. Needs no scaling
4. What happens as alpha in Ridge gets very large?
   - A. Nothing
   - B. Coefficients shrink towards zero and the model can underfit
   - C. The model becomes more complex
   - D. Training stops
5. Why must features be scaled before Ridge or Lasso?
   - A. So the penalty treats every feature fairly
   - B. Because Lasso needs text
   - C. It is not needed
   - D. To make training slower
6. In LogisticRegression, what does a small C mean?
   - A. Weak regularisation
   - B. Strong regularisation
   - C. A high threshold
   - D. No model

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.